La documentazione di rsedit

Dove sta ogni informazione, come si scrive, come si controlla

Indice

1 In breve 2
2 Le regole 3
2.1 Chi legge che cosa 3
2.2 Dove sta ogni informazione 3
2.3 I documenti 4
2.4 I manpage 4
2.5 Le docstring 5
2.6 Il rustdoc 6
2.7 I commenti nel codice 6
2.8 Il manuale per sviluppatori 7
2.9 Il rapporto sui problemi 7
2.10 La storia e i commit 7
2.11 I test 7
2.12 Le copie controllate 8
2.13 La lista di controllo di ogni patch 8
2.14 Tre esempi 8

1 In breve

La stessa informazione su rsedit potrebbe stare in più posti: nei commenti del codice, nelle docstring che `C-h f` mostra, nei manpage di `man/`, nei documenti di `doc/`. Copie che non si aggiornano insieme divergono. Questo documento dice dove sta ogni informazione, come si scrive e come si controlla che non diverga; vale per ogni modifica.

Le regole, in sei righe.

1. **Ogni fatto ha una casa sola.** Altrove si rimanda, non si copia.
2. **Chi usa l'editor legge i manpage**, completi e con esempi pratici. La docstring è il promemoria breve che `C-h f` mostra, e porta alla pagina.
3. **Chi sviluppa legge il manuale typst**: architettura, thread, lock e decisioni, con diagrammi. Il rustdoc è il contratto breve di ogni elemento.
4. **Un commento dice solo ciò che il codice non può dire**: un'invariante, un ordine di lock, un perché non ovvio, un TODO.
5. **Nel codice c'è solo il presente.** La storia sta nei commit, i difetti nel rapporto sui problemi.
6. **Una copia è ammessa solo se un test la confronta con la sua fonte.**

Il § 2 le spiega una per una; la lista di controllo di ogni modifica è in § 2.13, e § 2.14 mostra tre testi prima e dopo il riordino che le ha applicate.

2 Le regole

2.1 Chi legge che cosa

Chi	Dove guarda	Che cosa cerca
Chi usa l'editor	i manpage: `C-h m`, `M-x manpage`	come si fa una cosa, con esempi da copiare; i tasti; i limiti
Chi configura l'editor o scrive moduli Lisp	i manpage e le docstring: `C-h f`, `C-h v`	in più, la firma esatta di una funzione e il nome di una variabile
Chi sviluppa il Rust	il manuale in `doc/`, il rustdoc, i commenti	come è fatto il codice, perché, dove intervenire
Chi indaga un difetto	`doc/rapporto-problemi.typ` `git log`	che cosa è già noto, quando un comportamento è cambiato

2.2 Dove sta ogni informazione

Informazione	Casa	Altrove
Che cosa fa una funzione, un comando o una variabile: firma, argomenti, valore	la sua docstring	la SYNOPSIS della sua pagina la elenca; `C-h f` aggiunge il rimando alla pagina
Come si usa una funzionalità: concetti, combinazioni, esempi, limiti	il manpage del suo argomento	nessuna copia nelle docstring
I tasti	i keymap dei moduli Lisp	le sezioni KEYS delle pagine, controllate da un test; `C-h b` li calcola
I moduli caricati e il loro ordine	`DEFAULT_INIT_LISP` in `editor/boot.rs`	la sezione MODULES di `rsedit.txt`, controllata; `build.rs` e i test non ne tengono una propria
L'elenco delle pagine	la directory `man/`	l'indice di `guide.txt`, controllato; i test leggono la directory
Dove l'editor tiene i suoi file: configurazione, cache, log	il codice, `paths.rs`	la pagina `configuration`, per l'utente
La struttura: moduli, tipi principali, thread, lock, flussi	il manuale per sviluppatori, `doc/architettura-editor.typ`	la documentazione di modulo: poche righe e il rimando
Perché il codice è fatto così	le decisioni del capitolo del manuale	nel codice, al più una riga e il rimando
Invarianti, ordine dei lock, `SAFETY`, il thread su cui gira una funzione	il commento o il rustdoc accanto al codice	il manuale riassume l'ordine dei lock dell'intero editor
Il contratto di un elemento Rust: che cosa fa, errori, panic	il suo rustdoc	—
La storia: che cosa c'era prima e perché è cambiato	il messaggio di commit	ciò che ne resta valido è una decisione del manuale

Informazione	Casa	Altrove
I difetti noti e il lavoro rimandato	una scheda di `rapporto-problemi.typ`	`TODO(ID)` nel codice, finché la scheda è aperta
Il comportamento atteso, caso per caso	i test: il nome dice il comportamento	—
Come si misurano le prestazioni	`PERFORMANCE.md`	—
Che cos'è rsedit, come si compila, quali documenti ci sono e per chi	`README.md`	per tutto il resto rimanda ai manpage e ai documenti

`todo.org` è la lista personale dell'autore e resta fuori da queste regole.

2.3 I documenti

- `README.md` descrive rsedit in modo essenziale, secondo le pratiche di GitHub: che cos'è, come si compila, quali documenti ci sono e per chi.
- `man/README.md` dice com'è fatta una pagina, con il modello di § 2.4.
- In `doc/` stanno i documenti per chi sviluppa, in italiano e in Typst: il manuale (`architettura-editor.typ`), il rapporto sui problemi (`rapporto-problemi.typ`) e queste regole. Gli aiuti grafici comuni sono in `style.typ`. Accanto a ogni documento c'è il suo PDF, in git, rigenerato nello stesso commit della sorgente.
- Il linguaggio ha il suo manuale e il suo rapporto nel repository `risp`.

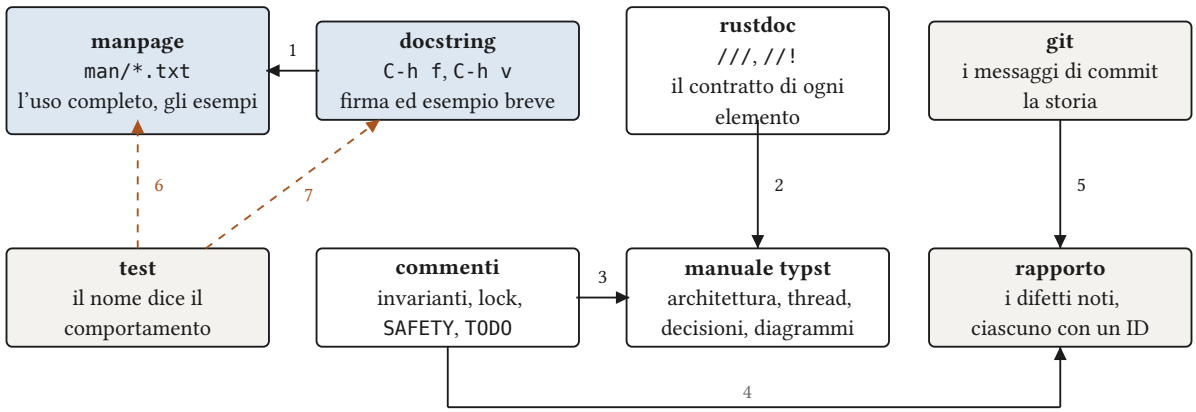


Figura 1: Le fonti e i loro rimandi. Frecce piene: «rimanda a»; tratteggiate: «controlla». In azzurro ciò che legge l'utente. (1) `C-h f` aggiunge «Manual:» con la pagina che elenca il nome nella sua SYNOPSIS. (2) La documentazione di modulo rimanda al capitolo, con un'etichetta. (3) Un perché lungo diventa una decisione del manuale. (4) `TODO(ID)` rimanda alla scheda. (5) Il commit che corregge un difetto chiude la sua scheda. (6, 7) I test confrontano le copie con le fonti e i nomi citati con quelli definiti.

2.4 I manpage

Una pagina per argomento. Le pagine sono il riferimento completo per chi usa l'editor: ogni nome pubblico — funzione, comando, variabile, modo — ha una pagina-casa che lo elenca nella SYNOPSIS e ne spiega l'uso. Una docstring non contiene niente che manchi alla pagina.

- **SYNOPSIS** elenca i nomi di cui la pagina è la casa: `(nome ARGOMENTI)` per funzioni e comandi, il nome da solo per le variabili. Ogni nome pubblico compare nella SYNOPSIS di una sola pagina. Da qui `C-h f` e `C-h v` ricavano il rimando «Manual:», che quindi non si scrive a mano.
- **EXAMPLES** c'è in ogni pagina: esempi completi, che si possono incollare in `init.lisp` o in `M-:`, ciascuno preceduto da una frase che dice a che cosa serve.
- **KEYS** e **KEYS IN … MODE** elencano i tasti: le controlla `guide_tests`.
- Le sezioni intermedie seguono ciò che il lettore vuole fare, non la struttura del codice. Il funzionamento interno va nel manuale per sviluppatori; la pagina dice ciò che l'utente vede e può fare, limiti compresi.
- In inglese, righe entro 78 colonne, intestazioni in maiuscolo a colonna 0 e corpo rientrato di quattro spazi: è ciò che `manpage-mode` riconosce e colora.

Il modello:

```
NAME
    dired - a directory in a buffer

SYNOPSIS
    (dired DIRECTORY)
    (dired-find-recursive)
    dired-sort
    find-file-recursive-limit

DESCRIPTION
    What it is for, and the idea behind it, in a few paragraphs.

OPENING A DIRECTORY
    One heading for each thing the reader wants to do.

EXAMPLES
    A sentence saying what the example is for, then the example,
    complete and ready to paste into init.lisp or M-:.

        (setq dired-sort 'type)        ; directories first

KEYS
    C-x d                   dired

KEYS IN DIRED MODE
    <ret>                   dired-find-file

SEE ALSO
    files, buffers
```

2.5 Le docstring

Valgono per le primitive (le costanti `*_DOC` di `primitives/`) e per le funzioni, i comandi e le variabili dei moduli Lisp. Seguono le convenzioni di Emacs.

```
(name ARGS): One sentence, in the imperative, saying what it does.

At most one short paragraph: what the arguments are, what comes back,
what is refused.

Example:
  (name ...)
```

- La prima riga basta da sola: è ciò che mostrano gli elenchi e i completamenti. Per le primitive comincia con la chiamata, `(name ARGS):`, come `doc_faithfulness_tests` già controlla.
- Gli argomenti in maiuscolo; gli altri nomi citati come `` `name' ``, la convenzione di Emacs: è così che il test dei nomi citati li riconosce.
- Un esempio quando l'uso non è ovvio.
- Di norma non più di 600 caratteri; un test rifiuta quelle oltre 1 000.
- Niente architettura, storia o motivazioni lunghe: vanno nella pagina o nel manuale. Alla pagina porta il rimando che `C-h f` aggiunge da sé.
- Le funzioni private, con `--` nel nome, hanno una riga che dice che cosa restituiscono. Un ragionamento interno è un commento `;;`, non una docstring.
- Le impostazioni che il Rust legge — variabili che nessun modulo dichiara con `defvar` — hanno la docstring nel registro `SETTINGS` di `editor/settings.rs`, che all'avvio la mette dove la mette `defvar`; il loro valore iniziale si imposta in `create_global_env`, così `C-h v` mostra anche quello. La docstring non ripete il valore iniziale.
- Un'impostazione di un modulo si dichiara con `defvar` e la sua docstring, non con un `setq` commentato: così `C-h v` la spiega e i test la vedono.

2.6 Il rustdoc

Segue le Rust API Guidelines e le convenzioni della RFC 1574.

- **La riga di riassunto** è una frase alla terza persona («Returns …», «Creates …») che basta da sola: è ciò che `cargo doc` mostra negli elenchi.
- Poi, se serve, un paragrafo breve. Per gli elementi dell'interfaccia pubblica (i `pub use` di `lib.rs`): le sezioni `# Errors`, `# Panics` e `# Safety` quando si applicano, e `# Examples`.
- I tipi e le funzioni citati sono link (`` [`Log::flush`] ``) e devono risolversi: `cargo doc --document-private-items` senza avvisi.
- Campi e varianti: una riga quando il nome non basta, niente quando basta.
- **La documentazione di modulo** (`//!`) dice in una-cinque righe di che cosa il modulo è responsabile, e rimanda al capitolo del manuale con la sua etichetta: `Design: `doc/architettura-editor.typ, <sfondo>`.`. Il rimando sta tra apici inversi perché il rustdoc, senza, legge `<sfondo>` come un tag HTML e dà un avviso. Un test controlla che l'etichetta esista.
- In inglese.

2.7 I commenti nel codice

Restano	Escono
• un'invariante su cui il codice conta e che i tipi non esprimono; • l'ordine in cui si prendono i lock, il thread su cui una funzione gira; • `SAFETY:` prima di ogni `unsafe`;	• ciò che il codice dice già; • la storia: «used to», «was … until», «the bug this fixes»; • le motivazioni lunghe, le sezioni `# Why`: diventano decisioni del manuale; • le spiegazioni per l'utente: vanno nel manpage;

Restano	Escono
• un perché non ovvio, in una o due righe, con il rimando alla decisione se ne serve di più; • il motivo di un aggiramento: un terminale, un sistema operativo; • `TODO(ID): …`, dove ID è una scheda del rapporto o un'etichetta come quelle già in uso (`uncertain`, `improve`).	• i riferimenti a cose che non ci sono più; • gli ID di problemi già risolti.

Nei moduli Lisp l'intestazione `;;;` dice in una-cinque righe che cosa il modulo aggiunge, e rimanda alla sua pagina e al capitolo del manuale.

2.8 Il manuale per sviluppatori

- Un solo manuale, `doc/architettura-editor.typ`, con un capitolo per sottosistema. Ogni capitolo dice di che cosa il sottosistema è responsabile e ne descrive le strutture dati principali, i thread e i lock, i flussi (con diagrammi), come si estende (le ricette) e le decisioni.
- Una decisione è una voce breve: il problema, la scelta, che cosa comporta, l'alternativa scartata e perché. Prende il posto delle sezioni `# Why` del codice.
- I diagrammi usano gli aiuti di `style.typ`. Sono obbligatori almeno per gli strati e le dipendenze dei moduli, i thread, l'ordine dei lock, il percorso da un tasto a un comando, il lavoro asincrono e la fotografia del frame.
- Il codice si cita per nome di funzione o di tipo, non per numero di riga, che invecchia a ogni modifica. Gli estratti sono brevi.
- Le etichette (`<sfondo>`) non cambiano, perché il codice vi rimanda.
- In italiano. Il PDF si rigenera nello stesso commit della sorgente.

2.9 Il rapporto sui problemi

- Una scheda per difetto, con un ID stabile.
- Il campo `stato`: `aperto` oppure `risolto`, con il commit che lo risolve. Le schede risolte escono dal riepilogo e passano in un'appendice compatta.
- Il codice cita un ID solo in un `TODO(ID)`, e solo finché la scheda è aperta.

2.10 La storia e i commit

Un commento che racconta la storia si cancella. Prima si guarda se contiene una decisione ancora valida: se sì, la decisione va nel manuale. Il resto della storia sta nel messaggio di commit, che dice che cosa cambia, perché, che cosa c'era prima e quale scheda chiude.

> Per questo le patch si applicano con `git am`, che conserva il loro messaggio di commit.

2.11 I test

- Il nome di un test descrive il comportamento, come già si fa: `a_variable_set_but_empty_counts_as_not_set`.
- Un commento in un test spiega in una o due righe un caso non ovvio. La storia del difetto che il test previene va nel commit.
- I test di documentazione sono quelli che controllano le copie (§ 2.12).

2.12 Le copie controllate

Copia	Fonte	Test
le sezioni KEYS	i keymap	`guide_tests`
i SEE ALSO	la directory `man/` e le funzioni	`guide_tests`
l'indice di `guide.txt`	la directory `man/`	`guide_tests`
la prima riga delle docstring delle primitive	il nome della primitiva	`doc_faithfulness_tests`
la sezione MODULES di `rsedit.txt`	`DEFAULT_INIT_LISP`	`guide_tests`
le SYNOPSIS	i nomi definiti	`guide_tests`
i nomi citati in pagine e docstring	i nomi definiti	`doc_faithfulness_tests`
i rimandi dal codice e dai test al manuale	le etichette typst	`doc_pointer_tests`
l'intestazione di ogni modulo Lisp	il nome del file, la pagina, l'etichetta	`doc_pointer_tests`
i rimandi del manuale al rapporto	le schede aperte	`doc_pointer_tests`
gli estratti del manuale	i file sorgente	`doc_pointer_tests` (che il file esista)

Un test non ammette eccezioni, tranne una: i nomi che pagine e docstring citano e che non sono funzioni né variabili dell'editor, e non devono esserlo — le forme speciali del Lisp, gli hook, i simboli che una funzione prende o restituisce, i nomi di Emacs dove il testo confronta i due —, elencati in `core/src/tests/doc_exceptions/quoted_elsewhere.txt`. Un nome che nessuno cita più si toglie dall'elenco.

2.13 La lista di controllo di ogni patch

> - Comportamento visibile cambiato ⟶ la pagina, con i suoi esempi.
> - Nome pubblico nuovo ⟶ la docstring e la SYNOPSIS della sua pagina.
> - Struttura, thread o lock cambiati ⟶ il capitolo del manuale e il suo diagramma.
> - Una scelta non ovvia ⟶ una voce tra le decisioni.
> - Difetto corretto ⟶ la scheda passa a `risolto` e il `TODO(ID)` si toglie.
> - Nessun commento storico, nessun rimando a cose tolte.
> - Test verdi, compresi quelli di documentazione; `cargo doc` senza avvisi; i PDF rigenerati.

2.14 Tre esempi

2.14.1 Una docstring: `background-call`

Prima del riordino la docstring era lunga 3 145 caratteri. Eccone l'inizio:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Do JOB in
the background under the name NAME, and return NAME at once without waiting
for it.

This is how anything that may take longer than a frame is started. JOB is
either a function of no arguments -- called once, in the background -- or a
`fiber', which is resumed a step at a time and is the shape to use when the
work can be done in pieces:
  [un esempio di nove righe e altri otto paragrafi]
```

Quasi tutto ripete la pagina `background`, che ha lo stesso esempio e le stesse regole. Dopo il riordino:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Run JOB in
the background under NAME, and return NAME at once. JOB is a function of no
arguments, called once, or a `fiber', resumed a step at a time as
`define-worker' resumes one. ON-PROGRESS is called with NAME after each step
but the last, and ON-DONE with NAME and t when JOB reached its end, or nil
when it failed; both run between commands. A job started under a name in use
replaces the one running, whose callbacks are then not called. INTERVAL is how
many milliseconds a plain (yield) waits: 40 unless given.

Example:
(defvar my-sum nil)
(defun my-done (name ok) (message "%s finished: %s" name ok))
(background-call 'my-sum (lambda () (setq my-sum (my-slow-sum))) nil 'my-done)

Manual: background
```

L'ultima riga non è scritta nella docstring: la aggiunge `C-h f`, dalla SYNOPSIS della pagina.

2.14.2 Un rustdoc: `app_dir`

Prima:

```
/// The directory OPTIONS asks for, where this system keeps it -- made first,
/// when it is missing and OPTIONS says to. See the module documentation for
/// where that is.
```

Dopo, con le sezioni della RFC 1574:

```
/// Returns the directory `options.dir` names on this system, creating it and
/// its parents first when it is missing and `options.create` is set.
///
/// Where each directory is: the `configuration` manual page.
///
/// # Errors
///
/// [`DirError::Undetermined`] when nothing in the environment says where the
/// directory is; [`DirError::NotFound`] when it is missing and `create` is
/// false; [`DirError::NotADirectory`] or [`DirError::CannotCreate`] otherwise.
///
/// # Examples
///
/// ```no_run
/// use rsedit_core::{AppDir, DirOptions, app_dir};
///
/// let cache = app_dir(DirOptions { dir: AppDir::Cache, create: true })?;
/// # Ok::<(), rsedit_core::DirError>(())
/// ```
```

La tabella delle precedenze, che era nella documentazione del modulo, è un'informazione per l'utente: è passata nella pagina `configuration`. Le due sezioni `# Why`, sul macOS e sul Windows, sono diventate decisioni del capitolo sui file dell'editor.

2.14.3 Una documentazione di modulo: `managers/log.rs`

Prima erano 29 righe:

```
//! Diagnostics: what has been logged, and where a copy of it goes.
//!
//! # Why these two are one thing
//!   [7 righe: la lista e il file si aggiornano insieme, e perché]
//! # Who writes the file
//!   [10 righe: append non scrive, flush sì, e lo chiama il watchdog]
//!
//! `log_diagnostic` used to take the list's lock and, still holding it,
//! `write_all` to the file, which put a disk write inside a lock every
//! diagnostic in the editor had to queue behind. The flush writes with the
//! list's lock given back.
```

Dopo:

```
//! Diagnostics: the list of what has been logged, and the file it is
//! copied to. [`Log::append`] only records; [`Log::flush`], which the
//! WorkManager's watchdog calls, writes what is owed.
//!
//! Design: `doc/architettura-editor.typ, <log>`.
```

Il primo `# Why` è diventato la decisione «La lista e il file si aggiornano insieme», e chi scrive il file, su quale thread, è nel capitolo sui thread. La storia si è cancellata (§ 2.10).